

**DEPARTMENT OF ARTIFICIAL INTELLIGENCE
AND DATA SCIENCE**

INNOVATION IN TEACHING REPORT

**Implementation of Real-Time Clock (RTC) Using Verilog HDL
on Spartan-3 FPGA (XC3S400-PQ208) for Digital Design
Laboratory**

By

Mr. Durairaji V

Assistant Professor

Department of Artificial Intelligence and Data Science

St. Joseph's College of Engineering

Chennai - 600119

1. Innovation Type

Hardware-Based Experiential Learning using FPGA Prototyping

The innovation involves demonstrating a complete Real-Time Clock (RTC) system implemented using Verilog HDL on a Xilinx Spartan-3 FPGA (XC3S400-PQ208) development board. Students learn digital design concepts through real-time hardware execution rather than simulation alone.

2. Course Details

Particular	Details
Course Name	Digital Principles and Logic Design
Laboratory	Digital Design Laboratory
Topic	Real-Time Clock using FPGA
Platform	Xilinx Spartan-3 FPGA XC3S400-PQ208
Programming Language	Verilog HDL
Software Tool	Xilinx ISE Design Suite
Clock Frequency	4 MHz
Display Device	6 - Digit Seven Segment Display

3. Objectives

The innovative teaching activity aims to:

1. Demonstrate real-time digital system design.
2. Introduce FPGA-based hardware implementation.
3. Explain clock division and timing circuits.
4. Illustrate counter design using Verilog HDL.
5. Familiarize students with synthesis, simulation, and hardware verification.
6. Bridge the gap between theoretical digital electronics and industrial FPGA applications.

4. Need for Innovation

Traditional teaching methods explain counters, registers, and timing circuits theoretically. Students often find it difficult to visualize how these concepts operate in real-time.

The FPGA-based RTC model provides:

Hands-on learning experience.

- ✓ Real-time observation of clock operation.
- ✓ Understanding of hierarchical digital design.
- ✓ Exposure to industrial FPGA technology.

- ✓ Improved problem-solving and debugging skills.

5. Description of the Innovative Model

The RTC system was designed using Verilog HDL and implemented on the Spartan-3 FPGA board.

The system:

- Accepts a 4 MHz clock input.
- Generates a 1-second pulse using a clock divider.
- Maintains:
 - ✓ Seconds Counter (00–59)
 - ✓ Minutes Counter (00–59)
 - ✓ Hours Counter (00–23)
- Displays time on a six-digit seven-segment display.
- Resets automatically after 23:59:59.

The implementation uses:

- Cascaded BCD Counters
- Clock Divider
- Seven Segment Decoder
- Display Multiplexing Technique

The Verilog code creates a 1-second enable pulse from the 4 MHz clock and updates the counters sequentially.

PROGRAM:

```
module finalclock_verilog(sgout,dis,load,reset,clock);
output [7:0]sgout;
output [5:0]dis;
input reset,clock,load;
reg [3:0]sec1_rg,sec2_rg,min1_rg,min2_rg,hr1_rg,hr2_rg;
reg [21:0] pulsegen ;
reg [2:0]cnk2;
reg [3:0]mout;
reg [7:0]sgout_rg;
reg [5:0]dis_sig;
wire [3:0]sec1,sec2,min1,min2,hr1,hr2;
wire tc1,tc2,tc3,tc4,tc5,tc6,enable;
```

```

//reg tc,tc1,tc2,tc3,tc4,tc5,tc6,enable;
//reg sec1_rg(3:0),sec2_rg(3:0),min1_rg(3:0),min2_rg(3:0),hr1_rg(3:0);
//reg hr2_rg(3:0),pulsegen(21:0),sel(2:0);
//reg mout(3:0),sgout(7:0),ck1(22:0),cnk2(2:0);

//***** Pulse Generator *****
always@(posedge clock or posedge reset)
begin
if (reset)
pulsegen <= 22'b0;
else
begin
//if (pulsegen == 22'b11110100001001000000000)
if (pulsegen == 22'b111101)
pulsegen <= 22'b0;
else
pulsegen <= pulsegen + 1;
end
end
//Enable signal to generate 1-sec pulse for sec1 counter

//assign enable = (pulsegen == 22'b11110100001001000000000) ; //enable signal for sec1
counter

assign enable = (pulsegen == 22'b111101) ; //enable signal for sec1 counter
//***** Second_cntr1 *****
always@(posedge clock or posedge reset)
begin
if (reset )
sec1_rg <= 4'b0000;
else if (load) begin
sec1_rg <= 4'b0100;
end
end

```

```

        else
begin
    if (enable)
        begin
            if (sec1_rg == 4'b1001)
                sec1_rg <= 4'b0000;
            else
                sec1_rg <= sec1_rg + 1;
            end
        end
    end
end

assign sec1 = sec1_rg;

//-----tc1 signal to start sec2 counter-----

assign tc1 = (sec1_rg == 4'b1001) && (enable == 1);

//_***** Second_cntr2 *****

always@(posedge clock or posedge reset)
begin
    if (reset )
        sec2_rg <= 4'b0000;

    else if (load) begin
        sec2_rg <= 4'b0100;
        end

    else
        begin
            if (tc1)
                begin
                    if (sec2_rg == 4'b0101) begin

```

```

                                sec2_rg <= 4'b0000;
                                end

    else begin
        sec2_rg <= sec2_rg + 1;
        end
    end
end

assign sec2 = sec2_rg;

//-----tc2 signal to start min1 counter-----
assign tc2 = (sec2_rg == 4'b0101) && (tc1 == 1);

//_***** Minute_cntr1 *****
always@(posedge clock or posedge reset)
begin
    if (reset )
        min1_rg <= 4'b0000;

    else if (load) begin
        min1_rg <= 4'b0100;
        end

    else
        begin
            if (tc2)
                begin
                    if (min1_rg == 4'b1001) begin
                        min1_rg <= 4'b0000;
                        end
                    else begin
                        min1_rg <= min1_rg + 1;
                        end
                end
        end
end

```

```

        end
    end
end

assign min1 = min1_rg;

//-----tc3 signal to start min2 counter-----
assign tc3 = (min1_rg == 4'b1001) && (tc2 == 1);

//_***** Hour_cntr1 *****
always@(posedge clock or posedge reset)
begin
    if (reset)
        min2_rg <= 4'b0000;

    else if (load) begin
        min2_rg <= 4'b0100;
        end

    else
        begin
            if (tc3)
                begin
                    if (min2_rg == 4'b0101) begin
                        min2_rg <= 4'b0000;
                        end

                    else
                        begin
                            min2_rg <= min2_rg + 1;
                            end
                        end
                end
        end
    end
end
end

```

```

assign min2 = min2_rg;

//-----tc4 signal to start hr1 counter-----
assign tc4 = (min2_rg == 4'b0101) && (tc3 == 1);

//_***** Hour_cntr1 *****
always@(posedge clock or posedge reset)
begin
if (reset)
hr1_rg <= 4'b0000;

else if (load) begin
                hr1_rg <= 4'b0001;
                end

else
begin
if (tc6)
begin
hr1_rg <= 4'b0000;
end
else
begin
if (tc4)
begin
if (hr1_rg == 4'b1001) begin
                hr1_rg <= 4'b0000;
                end

                end
            else
                begin
hr1_rg <= hr1_rg + 1;
                end
            end
        end
    end
end
end

```

```

    end
end

assign hr1 = hr1_rg;

//-----tc5 signal to start hr2 counter-----
assign tc5 = (hr1_rg == 4'b1001) && (tc4 == 1);

//-----tc6 signal to reset at 23:59:59-----
assign tc6 = (hr2_rg == 4'b0010) && (hr1_rg == 4'b0011) && (tc4 == 1);

//_***** Hour_cntr2 *****
always@(posedge clock or posedge reset)
begin
    if (reset)
        hr2_rg <= 4'b0000;
    else if (load) begin
        hr2_rg <= 4'b0000;
        end

    else
        begin
            if (tc6)
                begin
                    hr2_rg <= 4'b0000;
                    end
                else
                    begin
                        if (tc5)
                            begin
                                if (hr2_rg == 4'b0010) begin
                                    hr2_rg <= 4'b0000;
                                    end
                                end
                            else
                                begin
                                    end
                                end
                            end
                        end
                    end
                end
            end
        end
    end
end

```

```

        hr2_rg <= hr2_rg + 1;
        end

    end

end

end

end

assign hr2 = hr2_rg;

always@(posedge pulsegen[9] or posedge reset)
begin
    if (reset)
        cnk2 <= 3'b0;
    else
        begin
            if (cnk2 == 3'b101)
                cnk2 <= 3'b0 ;
            else
                cnk2 <= cnk2 + 1;
            end
        end
    end
always@(cnk2)
case (cnk2)
3'b000 : mout <= sec1;
3'b001 : mout <= sec2;
3'b010 : mout <= min1;
3'b011 : mout <= min2;
3'b100 : mout <= hr1;
3'b101 : mout <= hr2;
endcase
always@(mout)
case (mout)
4'b0000 : sgout_rg <= 8'b11000000;

```

```

4'b0001 : sgout_rg <= 8'b11111001;
4'b0010 : sgout_rg <= 8'b10100100;
4'b0011 : sgout_rg <= 8'b10110000;
4'b0100 : sgout_rg <= 8'b10011001;
4'b0101 : sgout_rg <= 8'b10010010;
4'b0110 : sgout_rg <= 8'b10000010;
4'b0111 : sgout_rg <= 8'b11111000;
4'b1000 : sgout_rg <= 8'b10000000;
4'b1001 : sgout_rg <= 8'b10011000;
endcase
always@(cnk2)
case (cnk2)
3'b000 : dis_sig <= 6'b111110;
3'b001 : dis_sig <= 6'b111101;
3'b010 : dis_sig <= 6'b111011;
3'b011 : dis_sig <= 6'b110111;
3'b100 : dis_sig <= 6'b101111;
3'b101 : dis_sig <= 6'b011111;
endcase
assign sgout = ~ sgout_rg;
assign dis = ~ dis_sig;
endmodule

```

6. Hardware Setup

- Spartan-3 FPGA Board (XC3S400-PQ208)
- 4 MHz Crystal Oscillator
- Six-Digit Seven Segment Display Board
- Ribbon Cable Interface
- Power Supply

7. FPGA Pin Assignments

Important pin mappings from the UCF (User Constraints File) file:

```

net clock loc = p181;
net reset loc = p182;
net load loc = p33;

```

```
net sgout<7> loc =p199;
net sgout<6> loc =p198;
net sgout<5> loc =p197;
net sgout<4> loc =p196;
net sgout<3> loc =p194;
net sgout<2> loc =p191;
net sgout<1> loc=p190;
net sgout<0> loc=p189;

net dis<5> loc = p10;
net dis<4> loc = p11;
net dis<3> loc = p12;
net dis<2> loc = p13;
net dis<1> loc = p15;
net dis<0> loc = p16;
```

8. Working Principle

Step 1: Clock Division

The FPGA receives a 4 MHz clock.

A pulse generator divides the clock and generates a 1-second enable signal.

Step 2: Seconds Counter

- Counts from 00 to 59.
- Generates carry signal tc1.

Step 3: Minutes Counter

- Increments whenever seconds reach 59.
- Counts from 00 to 59.

Step 4: Hours Counter

- Counts from 00 to 23.
- Resets automatically after 23:59:59.

Step 5: Display Multiplexing

The six seven-segment displays are activated one after another at high speed to create continuous visualization.

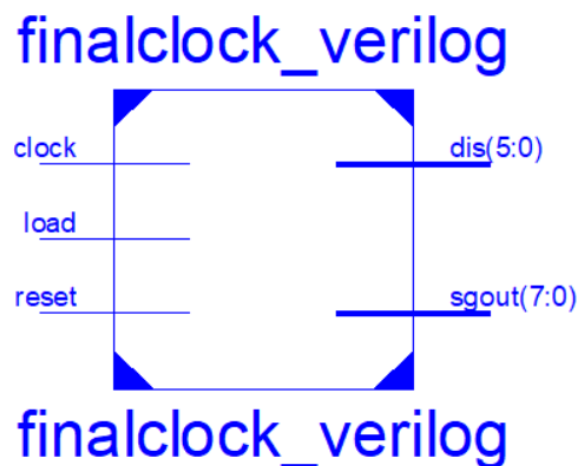
9. RTL (Register-Transfer Level) Schematic

The RTL design contains:

- Clock Divider Module
- Second Counter Module

- Minute Counter Module
- Hour Counter Module
- Seven Segment Decoder
- Display Multiplexer

The synthesized RTL schematic clearly shows the hierarchical implementation of the RTC design.

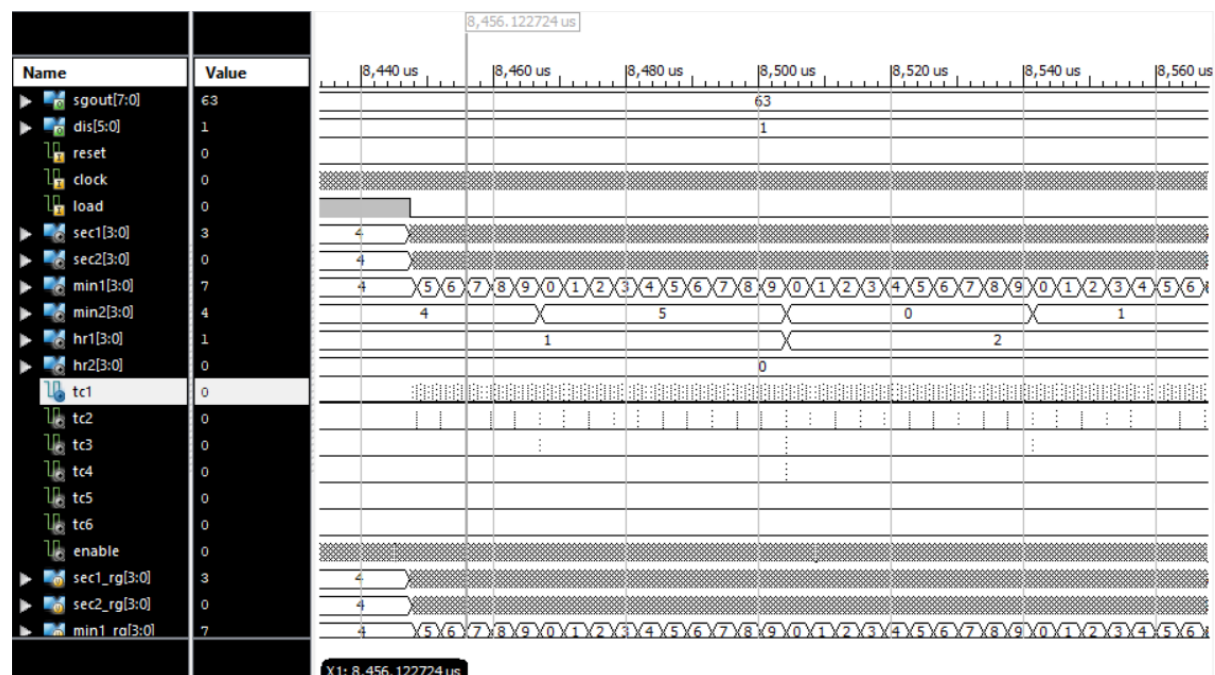


10. Simulation Results

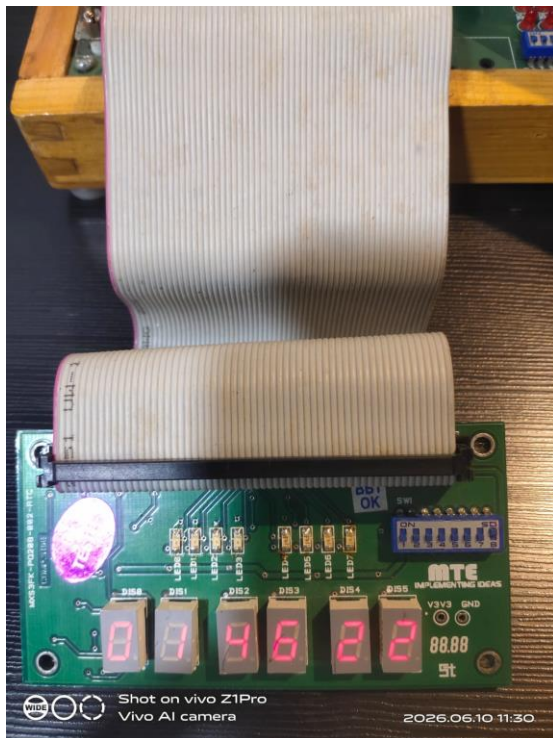
Observed outputs:

Time Display

01:47:03



11. Hardware Implementation



12. Learning Outcomes

After performing this activity, students are able to:

- ✓ Design synchronous sequential circuits.
- ✓ Implement clock dividers.
- ✓ Develop Verilog HDL modules.
- ✓ Perform FPGA synthesis and implementation.
- ✓ Analyze RTL schematics.
- ✓ Verify functionality using simulation tools.
- ✓ Interface FPGA with seven-segment displays.
- ✓ Develop real-time embedded digital systems.

13. Outcome-Based Education (OBE) Mapping

POs	Contribution
PO1	Engineering Knowledge
PO2	Problem Analysis
PO3	Design of Solutions
PO4	Investigation
PO5	Modern Tool Usage
PO12	Lifelong Learning

14. Conclusion

The Real-Time Clock (RTC) implementation using Verilog HDL on Spartan-3 FPGA XC3S400-PQ208 is an effective innovative teaching model that transforms theoretical concepts of counters, timing circuits, and sequential logic into a practical real-time hardware application. The activity enhances student engagement, improves FPGA design skills, and provides industrial-level exposure to digital system development.

Innovation Outcome: Students achieved a better understanding of digital design concepts through real-time FPGA implementation, simulation verification, RTL analysis, and hardware testing.